

DBMS WEEKLY TASK

PAYMENT MANAGEMENT SYSTEM

1. Objectives

The main objectives of this task are:

- To create a Payment table.
- To store customer payment details.
- To manage successful and failed transactions.
- To modify payment status when required.
- To analyze the different payment methods used.
- To calculate the amount collected through each payment mode.
- To generate a payment transaction report.

2. Payment Table Structure

Column	Data Type	Constraint	Description
PAYMENT_ID	NUMBER(5)	Primary Key	Unique payment identification number
ORDER_ID	NUMBER(5)	Foreign Key	Identifies the related order
PAYMENT_DATE	DATE	NOT NULL	Date of payment
PAYMENT_MODE	VARCHAR2(20)	NOT NULL	Payment method
PAYMENT_AMOUNT	NUMBER(10,2)	NOT NULL	Amount paid
PAYMENT_STATUS	VARCHAR2(20)	NOT NULL	Status of payment

3. Create Payment Table

- The Payment table is created to store all payment-related information. The PAYMENT_ID is the primary key, and ORDER_ID is connected to the Orders table using a foreign key.

```
payment_output.txt
SQL> CREATE TABLE PAYMENT (
2     PAYMENT_ID NUMBER(5) PRIMARY KEY,
3     ORDER_ID NUMBER(5),
4     PAYMENT_DATE DATE NOT NULL,
5     PAYMENT_MODE VARCHAR2(20) NOT NULL,
6     PAYMENT_AMOUNT NUMBER(10,2) NOT NULL,
7     PAYMENT_STATUS VARCHAR2(20) NOT NULL,
8     CONSTRAINT FK_PAYMENT_ORDER
9         FOREIGN KEY (ORDER_ID)
10            REFERENCES ORDERS(ORDER_ID)
11 );

Table created.
```

4. Insert Payment Details

Payment details are inserted into the Payment table using the INSERT command.

```
SQL> SELECT * FROM PAYMENT;
```

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
501	1001	22-SEP-26	UPI	1198	SUCCESS
502	1002	22-SEP-26	CARD	999	SUCCESS
503	1003	23-SEP-26	CASH	750	FAILED
504	1004	23-SEP-26	UPI	1500	SUCCESS
505	1005	24-SEP-26	CARD	850	FAILED

5. Display Successful Payments

Successful transactions can be displayed using the WHERE condition.

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
501	1001	22-SEP-26	UPI	1198	SUCCESS
502	1002	22-SEP-26	CARD	999	SUCCESS
504	1004	23-SEP-26	UPI	1500	SUCCESS

6. Display Failed Payments

Failed transactions can also be identified using the WHERE condition.

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
503	1003	23-SEP-26	CASH	750	FAILED
505	1005	24-SEP-26	CARD	850	FAILED

7. Modify Payment Status

The payment status can be modified when the transaction status changes.

PAYMENT_MODE	TOTAL_TRANSACTIONS
UPI	2
CARD	2
CASH	1

8. Analyze Payment Methods

The number of transactions made using each payment method can be calculated using GROUP BY and COUNT().

```
SQL> SELECT PAYMENT_MODE,
2         SUM(PAYMENT_AMOUNT) AS TOTAL_AMOUNT_COLLECTED
3   FROM PAYMENT
4  WHERE PAYMENT_STATUS = 'SUCCESS'
5  GROUP BY PAYMENT_MODE;
```

PAYMENT_MODE	TOTAL_AMOUNT_COLLECTED
UPI	2698
CARD	999

9. Payment Transaction Report

A payment transaction report can be generated by joining the Payment table with the Orders table.

PAYMENT_ID	ORDER_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
501	1001	22-SEP-26	UPI	1198	SUCCESS
502	1002	22-SEP-26	CARD	999	SUCCESS
503	1003	23-SEP-26	CASH	750	FAILED
504	1004	23-SEP-26	UPI	1500	SUCCESS
505	1005	24-SEP-26	CARD	850	FAILED